CSC 659/859 Spring 2026 - HW 2

ML Pipeline Experiment with Random Forest

Class: CSC 659/859
Semester: Spring 2026
Date: 3/12/26
Name: Nathaniel Moreno
Email: nmoreno@sfsu.edu

---

1. Title Page

HW 2 Title: ML Pipeline Experiment with Random Forest for i1 Cluster Classification
Class: CSC 659/859
Semester: Spring 2026
Date: [Insert Current Date]
Name: Nathaniel (Nate) Moreno
Email: nmoreno@sfsu.edu

---

2. Audit of Original Database

2.1 Data Source

The data used in this assignment is from the J. Craig Venter Institute (JCV) experiment, specifically focusing on i1 nerve cluster classification. The biological background and ground truth for the i1 cluster are established in the following research paper:

Reference: Aevermann B., Novotny M., Bakken T., Miller J., Diehl A., Osumi-Sutherland D., Lasken R., Lein E., Scheuermann R.: "Cell type discovery using single cell transcriptomics: implications for ontological representation", Human Molecular Genetics 27(R1): R40-R47 · March 2018

2.2 Database Audit Checklist

Key Code Snippet for Database Audit:

```python
# Load the original database
df_original = pd.read_csv('Original training DB i1 cluster.csv')
print(f"\nOriginal Database Statistics:")
print(f"  Total samples: {len(df_original)}")
print(f"  Total features: {len(df_original.columns) - 1}")  # Excluding label column
print(f"  Total columns: {len(df_original.columns)}")

# Check label distribution
label_counts = df_original['Label'].value_counts()
print(f"\nLabel Distribution:")
print(f"  Class 0 (non-i1): {label_counts[0]} samples ({label_counts[0]/len(df_original)*100:.2f}%)")
print(f"  Class 1 (i1 cluster): {label_counts[1]} samples ({label_counts[1]/len(df_original)*100:.2f}%)")

# Check for missing values
missing_values = df_original.isnull().sum().sum()
print(f"\nMissing Values: {missing_values}")

# Check data types
print(f"\nData Types:")
print(f"  All features are numerical: {df_original.iloc[:, :-1].dtypes.apply(lambda x: pd.api.types.is_numeric_dtype(x)).all()}")

# Check if class is unbalanced (less than 10% in minority class)
minority_class_pct = min(label_counts) / len(df_original) * 100
is_unbalanced = minority_class_pct < 10
print(f"\nClass Balance Check:")
print(f"  Minority class percentage: {minority_class_pct:.2f}%")
print(f"  Is unbalanced (<10%): {is_unbalanced}")

# Check sample-to-feature ratio
n_features = len(df_original.columns) - 1
n_samples = len(df_original)
ratio = n_samples / n_features
print(f"\nSample-to-Feature Ratio:")
print(f"  Ratio: {ratio:.2f}:1")
print(f"  Meets requirement (>=10:1): {ratio >= 10}")
```

2.2.1 Feature Description and Data Format

- **Features:** 608 gene expression levels (numerical values)
- **Class Label:** Binary classification (0 = non-i1, 1 = i1 cluster)
- **Data Format:** All features are numerical (continuous values representing gene expression levels)
- **Missing Values:** 0 (no missing values in the dataset)

2.2.2 Class Label Verification

The class labels are verified against ground truth established by independent biological research. The i1 cluster is defined as: "A human MTG cortical layer 1 GABAergic interneuron that selectively expresses COL5A2 and NDNF and FAT1 mRNAs" (Aevermann et al., 2018).

2.2.3 Demography Coverage

[Not applicable for this biological dataset - no demographic features]

2.2.4 Class Balance Analysis

Original Database Statistics: - Total samples: 871 - Class 0 (non-i1): 781 samples (89.67%) - Class 1 (i1 cluster): 90 samples (10.33%) - **Unbalanced class check:** No - minority class is 10.33%, which is slightly above the 10% threshold, so the dataset is not considered severely unbalanced

2.2.5 Feature Types

All features are numerical (continuous) representing gene expression levels. No categorical features present.

2.2.6 Missing Values

- **Missing values count:** 0
- **Handling:** No missing values were found in the dataset. According to the assignment instructions, a value of 0 for a feature means the gene expression level is 0, not that the value is missing. No imputation was needed.

2.2.7 Sample-to-Feature Ratio

- **Number of samples:** 871
- **Number of features:** 608
- **Ratio:** 1.43:1 (871/608 = 1.43)
- **Requirement met (≥10:1): No** - The ratio of 1.43:1 does NOT meet the requirement of at least 10:1 (which would require at least 6,080 samples).

Critical Analysis: The number of features (608) exceeds what would typically be recommended given the sample size. This is a significant limitation of the dataset. However, Random Forest is particularly well-suited for this high-dimensional scenario because: 1. Random Forest naturally handles high-dimensional data through feature subsampling (max_features parameter) 2. The ensemble approach with multiple trees helps reduce overfitting 3. Regularization parameters (max_depth=20, min_samples_split=5, min_samples_leaf=1) are used to prevent overfitting 4. The OOB score provides internal validation without needing a separate validation set

This limitation should be acknowledged, but Random Forest's inherent properties make it an appropriate choice for this dataset despite the unfavorable sample-to-feature ratio.

2.2.8 Privacy Issues

[Not applicable - no personally identifiable information in gene expression data]

2.3 Summary Statistics Table

Statistic	Value
Total Samples	871
Total Features	608
Class 0 Samples	781 (89.67%)
Class 1 Samples	90 (10.33%)
Missing Values	0
Sample-to-Feature Ratio	1.43:1
All Features Numerical	Yes
Class Unbalanced	No (minority class 10.33%)

---

3. Creation of Training DB and Verification DB

3.1 Methodology

From the original database, two random samples were selected: - One sample from class 1 (i1 cluster) - One sample from class 0 (non-i1)

These two samples were placed in the Verification Database. The remaining samples form the Training Database.

Random seed used: 42 (for reproducibility)

Key Code Snippet:

```python

# Randomly select 1 sample from each class for verification
np.random.seed(42)
verification_class_0_idx = np.random.choice(class_0_samples.index, size=1, replace=False)
verification_class_1_idx = np.random.choice(class_1_samples.index, size=1, replace=False)

# Create verification database
verification_db = pd.concat([
    df_original.loc[verification_class_0_idx],
    df_original.loc[verification_class_1_idx]
], ignore_index=True)

# Create training database (original minus verification samples)
training_db = df_original.drop(index=verification_class_0_idx).drop(index=verification_class_1_idx).reset_index(drop=True)

print(f"\nTraining Database Statistics:")
```

Data Split Code *Figure 2: Code snippet for splitting the original database into training and verification databases by randomly selecting one sample from each class.*

3.2 Training Database Statistics

Statistic	Value
Total Samples	869
Class 0 Samples	780
Class 1 Samples	89
Total Features	608

3.3 Verification Database Statistics

Statistic	Value
Total Samples	2
Class 0 Samples	1
Class 1 Samples	1
Total Features	608

---

4. SW Tools

4.1 Software Tools Used

- **Programming Language:** Python 3.x

- **ML Library:** scikit-learn (sklearn)
- **Data Processing:** pandas, numpy
- **Environment:** Python script (command line execution)

4.2 Key Libraries and Versions

- pandas: 2.3.3
- numpy: 2.3.5
- scikit-learn: 1.8.0

---

5. Experimental Methods and Setup

5.1 Method 1: Basic Training with OOB Accuracy Estimate

5.1.1 Method Description

Out-of-Bag (OOB) accuracy estimation is an internal validation method used by Random Forest. Each tree in the forest is trained on a bootstrap sample of the data, and the samples not included in a tree's bootstrap sample (the "out-of-bag" samples) are used to evaluate that tree's performance. The OOB score is the average accuracy across all trees using their respective OOB samples.

5.1.2 Software Methods/Classes Used

- `RandomForestClassifier` from `sklearn.ensemble`
- `ParameterGrid` from `sklearn.model_selection` for grid search
- `oob_score=True` parameter to enable OOB scoring

5.1.3 Hyperparameter Ranges

The following hyperparameter ranges were used in the grid search:

Hyperparameter	Values Tested
n_estimators	[200, 300, 500]
max_features	['sqrt', 0.3, 0.5]
max_depth	[20, 30, None]
min_samples_split	[2, 5]
min_samples_leaf	[1, 2]
class_weight	['balanced']

Total combinations tested: 108

Note: A reduced grid search was used for computational efficiency. The full grid would have included: n_estimators [100, 200, 300, 500], max_features ['sqrt', 'log2', 0.3, 0.5], max_depth [10, 20, 30, None], min_samples_split [2, 5, 10], min_samples_leaf [1, 2, 4], class_weight ['balanced', None], which would result in 1,152 combinations.

5.1.4 Accuracy Measures Computed

The following accuracy measures were computed from the training set predictions:

1. **OOB Accuracy:** Calculated automatically by RandomForestClassifier when `oob_score=True`
2. **Training Accuracy:** (TP + TN) / (TP + TN + FP + FN)
3. **Precision:** TP / (TP + FP)
4. **Recall (Sensitivity):** TP / (TP + FN)
5. **F1 Score:** 2 × (Precision × Recall) / (Precision + Recall)

Where: - TP = True Positives - TN = True Negatives - FP = False Positives - FN = False Negatives

5.2 Method 2: 3-Fold Cross-Validation

5.2.1 Method Description

3-fold cross-validation manually splits the training data into 3 equal (or nearly equal) folds. For each fold: 1. One fold is used as the test set 2. The remaining 2 folds are combined to form the training set 3. A Random Forest model is trained on the training set 4. The model is evaluated on the test set 5. Accuracy metrics are computed

The final CV accuracy is the average of the three fold accuracies.

5.2.2 Implementation Steps

The CV was implemented **manually** (NOT using sklearn's built-in CV methods like `cross_val_score`, `KFold`, or `cross_validate`) in the following steps:

1. **Shuffle the training data indices** (random seed: 42)
2. **Split indices into 3 folds manually:**
   - Fold 1: Indices [0:fold_size] → Used as Test Set
   - Fold 2: Indices [fold_size:2*fold_size] → Used as Test Set
   - Fold 3: Indices [2*fold_size:] → Used as Test Set
3. **For each fold (manually loop through):**
   - **Fold 1:** Train on Folds [2,3], Test on Fold [1]
   - **Fold 2:** Train on Folds [1,3], Test on Fold [2]
   - **Fold 3:** Train on Folds [1,2], Test on Fold [3]
   - For each fold:
     - Split data into train/test using the fold indices
     - **Report the number of Class 0 and Class 1 samples in each train/test partition**
     - Train Random Forest model on training partition
     - Predict on test partition
     - Compute accuracy, precision, recall, F1 score, and confusion matrix

4. **Compute average metrics** across all 3 folds

Important: This implementation does NOT use any ready-made CV methods. Each fold is explicitly split and processed manually in a loop.

5.2.3 Software Methods/Classes Used

- Manual fold splitting using numpy array indexing
- `RandomForestClassifier` from `sklearn.ensemble`
- `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `confusion_matrix` from `sklearn.metrics`

5.2.4 Accuracy Measures Computed

For each fold: - Accuracy - Precision - Recall - F1 Score - Confusion Matrix

Final CV metrics are the average of the three folds.

5.3 Verification and Comparison of Methods 1 and 2

The results from both methods will be compared based on: - Accuracy measures (OOB vs CV average accuracy) - F1 scores - Consistency of results - Computational efficiency considerations

---

6. Actual Results of RF Training and Accuracy Estimates

6.1 Method 1 Results: OOB Accuracy Estimation

6.1.1 Best Hyperparameters

Hyperparameter	Best Value
n_estimators	300
max_features	0.3
max_depth	20
min_samples_split	5
min_samples_leaf	1
class_weight	balanced

6.1.2 Key Code Snippet

```python
    print("\nPerforming grid search with OOB scoring...")
    for i, params in enumerate(ParameterGrid(reduced_param_grid)):
        if i % 10 == 0:
            print(f"  Testing combination {i+1}/{len(list(ParameterGrid(reduced_param_grid)))}...")

        # Create RF with OOB scoring enabled
        rf = RandomForestClassifier(
            n_estimators=params['n_estimators'],
            max_features=params['max_features'],
            max_depth=params['max_depth'],
            min_samples_split=params['min_samples_split'],
            min_samples_leaf=params['min_samples_leaf'],
            class_weight=params['class_weight'],
            oob_score=True,
            random_state=42,
            n_jobs=-1
        )

        # Train the model
        rf.fit(X_train, y_train)

        # Get OOB score
        oob_score = rf.oob_score_

        # Store results
        results_method1.append({
            'params': params.copy(),
            'oob_score': oob_score,
            'model': rf
        })

        # Update best model
        if oob_score > best_oob_score:
            best_oob_score = oob_score
            best_params = params.copy()
            best_rf_oob = rf
```

Method 1 OOB Code *Figure 3: Code snippet for Method 1 - OOB accuracy estimation with grid search. Shows the RandomForestClassifier setup with OOB scoring enabled and the grid search loop.*

6.1.3 Confusion Matrix (Training Set)

Table 1: Confusion Matrix for Method 1 (Training Set) *This confusion matrix shows the classification results on the training set using the best OOB model. Rows represent true labels, columns represent predicted labels.*

	Predicted: 0	Predicted: 1
True: 0	780	0
True: 1	0	89

6.1.4 All Accuracy Measures

Code Snippet for Computing Accuracy Measures:

```python

# Get predictions on training set for confusion matrix
y_train_pred = best_rf_oob.predict(X_train)
cm_train = confusion_matrix(y_train, y_train_pred)

# Calculate all accuracy measures
accuracy_train = accuracy_score(y_train, y_train_pred)
precision_train = precision_score(y_train, y_train_pred, zero_division=0)
recall_train = recall_score(y_train, y_train_pred, zero_division=0)
f1_train = f1_score(y_train, y_train_pred, zero_division=0)

```

Accuracy Measures Code *Figure 4: Code snippet for computing all accuracy measures including accuracy, precision, recall, F1 score, and confusion matrix from the best OOB model.*

Table 2: Method 1 Accuracy Measures *All accuracy measures computed from the best OOB model on the training set. All values shown to 5 decimal places.*

Metric	Value
OOB Accuracy	0.99310
Training Accuracy	1.00000
Precision	1.00000
Recall	1.00000
F1 Score	1.00000

6.2 Method 2 Results: 3-Fold Cross-Validation

6.2.1 Fold 1 Results

Train/Test Split: - Train: 580 samples (Class 0: 526, Class 1: 54) - Test: 289 samples (Class 0: 254, Class 1: 35)

Metrics: | Metric | Value | |———–|——-| | Accuracy | 0.99308 | | Precision | 0.97143 | | Recall | 0.97143 | | F1 Score | 0.97143 |

Confusion Matrix: | | Predicted: 0 | Predicted: 1 | |—|—|—| | **True: 0** | 253 | 1 | | **True: 1** | 1 | 34 |

6.2.2 Fold 2 Results

Train/Test Split: - Train: 580 samples (Class 0: 513, Class 1: 67) - Test: 289 samples (Class 0: 267, Class 1: 22)

Metrics: | Metric | Value | |———–|——-| | Accuracy | 0.99308 | | Precision | 1.00000 | | Recall | 0.90909 | | F1 Score | 0.95238 |

Confusion Matrix: | | Predicted: 0 | Predicted: 1 | |—|—|—| | **True: 0** | 267 | 0 | | **True: 1** | 2 | 20 |

6.2.3 Fold 3 Results

Train/Test Split: - Train: 578 samples (Class 0: 521, Class 1: 57) - Test: 291 samples (Class 0: 259, Class 1: 32)

Metrics: | Metric | Value | |———–|——-| | Accuracy | 0.98625 | | Precision | 1.00000 | | Recall | 0.87500 | | F1 Score | 0.93333 |

Confusion Matrix: | | Predicted: 0 | Predicted: 1 | |—|—|—| | **True: 0** | 259 | 0 | | **True: 1** | 4 | 28 |

6.2.4 Final CV Results (Average)

Table 3: Method 2 Average CV Metrics *Average accuracy measures across all 3 folds. All values shown to 5 decimal places.*

Metric	Average Value
Accuracy	0.99080
Precision	0.99048
Recall	0.91851
F1 Score	0.95238

6.2.5 Key Code Snippet

```python
# Shuffle the training data
train_indices = np.arange(len(training_db))
np.random.seed(42)
np.random.shuffle(train_indices)

# Split into 3 folds manually
n_samples = len(train_indices)
fold_size = n_samples // 3
folds = [
    train_indices[0:fold_size],
    train_indices[fold_size:2*fold_size],
    train_indices[2*fold_size:]
]

print(f"\nFold sizes: {[len(fold) for fold in folds]}")

cv_results = []
best_cv_score = 0
best_rf_cv = None
best_cv_params = None

# Use best parameters from Method 1 as starting point, or use a fixed set
# For CV, we'll use the best params from Method 1
cv_params = best_params.copy()

for fold_num in range(3):
    print(f"\n--- Fold {fold_num + 1} ---")

    # Create test and train indices for this fold
    test_indices = folds[fold_num]
    train_indices_fold = np.concatenate([folds[i] for i in range(3) if i != fold_num])

    # Split data
    X_train_fold = X_train.iloc[train_indices_fold]
    y_train_fold = y_train.iloc[train_indices_fold]
    X_test_fold = X_train.iloc[test_indices]
    y_test_fold = y_train.iloc[test_indices]

    print(f"  Train: {len(X_train_fold)} samples (Class 0: {sum(y_train_fold==0)}, Class 1: {sum(y_train_fold==1)})")
    print(f"  Test: {len(X_test_fold)} samples (Class 0: {sum(y_test_fold==0)}, Class 1: {sum(y_test_fold==1)})")

    # Train RF model
    rf_cv = RandomForestClassifier(
        n_estimators=cv_params['n_estimators'],
        max_features=cv_params['max_features'],
        max_depth=cv_params['max_depth'],
        min_samples_split=cv_params['min_samples_split'],
        min_samples_leaf=cv_params['min_samples_leaf'],
        class_weight=cv_params['class_weight'],
        random_state=42,
         random_state=42,
         n_jobs=-1
     )

     rf_cv.fit(X_train_fold, y_train_fold)

     # Predict on test set
     y_pred_fold = rf_cv.predict(X_test_fold)

     # Calculate metrics
     accuracy_fold = accuracy_score(y_test_fold, y_pred_fold)
     precision_fold = precision_score(y_test_fold, y_pred_fold, zero_division=0)
     recall_fold = recall_score(y_test_fold, y_pred_fold, zero_division=0)
     f1_fold = f1_score(y_test_fold, y_pred_fold, zero_division=0)
     cm_fold = confusion_matrix(y_test_fold, y_pred_fold)

     print(f"  Accuracy: {accuracy_fold:.5f}")
     print(f"  Precision: {precision_fold:.5f}")
     print(f"  Recall: {recall_fold:.5f}")
     print(f"  F1 Score: {f1_fold:.5f}")
     print(f"  Confusion Matrix: TN={cm_fold[0,0]}, FP={cm_fold[0,1]}, FN={cm_fold[1,0]}, TP={cm_fold[1,1]}")
```

Method 2 CV Code *Figure 5: Code snippet for Method 2 - Manual 3-fold cross-validation implementation. Shows manual fold splitting, training, prediction, and metric computation for each fold.*

6.3 Comparison of Method 1 and Method 2

Table 4: Comparison of Method 1 (OOB) and Method 2 (3-Fold CV) *Comparison of accuracy measures from both methods. All values shown to 5 decimal places.*

Metric	Method 1 (OOB)	Method 2 (CV)	Difference
Accuracy	0.99310	0.99080	0.00230
F1 Score	1.00000	0.95238	0.04762

Discussion:

Both methods show excellent performance, with Method 1 (OOB) achieving slightly higher accuracy (0.99310 vs 0.99080) and a perfect F1 score (1.00000 vs 0.95238). The difference in F1 score (0.04762) is more noticeable than the accuracy difference (0.00230), suggesting that Method 1 has better balance between precision and recall.

Method 1 (OOB) Strengths: - Higher accuracy and perfect F1 score on training data - More computationally efficient (no need to train multiple models) - Uses all available training data for the final model - OOB score provides an unbiased estimate without separate validation set

Method 1 (OOB) Weaknesses: - OOB score is computed on samples not used in bootstrap, but the model still sees all training data - May be slightly optimistic compared to true hold-out validation

Method 2 (CV) Strengths: - Provides more robust estimate by testing on completely held-out data - Shows performance variability across different data splits - Better for understanding model generalization - Average metrics across folds give more stable estimates

Method 2 (CV) Weaknesses: - More computationally expensive (trains 3 separate models) - Lower F1 score suggests slightly less balanced performance - Each fold uses less training data (2/3 of data vs full data in Method 1)

Conclusion: Both methods indicate excellent model performance. Method 1's perfect training performance suggests the model fits the training data very well, while Method 2's slightly lower but still excellent performance (0.99080 accuracy, 0.95238 F1) provides a more conservative estimate of generalization. For this dataset, both methods validate that the Random Forest model is performing well. The choice between them depends on whether computational efficiency (Method 1) or more robust validation (Method 2) is prioritized.

---

7. Feature Ranking

7.1 Methodology

Feature ranking was performed using the **Gini importance** method (default in scikit-learn's RandomForestClassifier). The feature importance is calculated as the total decrease in node impurity (weighted by the probability of reaching that node) averaged over all trees in the forest.

Model Used: Best trained RF model from Method 1

7.2 Key Code Snippet

```python
# Use the best model from Method 1 for feature ranking
rf_for_ranking = best_rf_oob

# Get feature importances (Gini importance)
feature_importances = rf_for_ranking.feature_importances_
feature_names = X_train.columns

# Create Dataframe for easier handling
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances
}).sort_values('Importance', ascending=False)

print(f"\nTop 10 Features by Gini Importance:")
print(importance_df.head(10).to_string(index=False))

# Check for ground truth genes (COL5A2, NDNF, FAT1)
ground_truth_genes = ['COL5A2', 'NDNF', 'FAT1']
print(f"\nGround Truth Gene Analysis:")
for gene in ground_truth_genes:
    # Find all features containing this gene name
    matching_features = importance_df[importance_df['Feature'].str.contains(gene, case=False, na=False)]
    if len(matching_features) > 0:
        print(f"\n  {gene} family features found:")
        for idx, row in matching_features.iterrows():
            rank = importance_df.index.get_loc(idx) + 1
            print(f"    {row['Feature']}: Rank {rank}, Importance {row['Importance']:.5f}")
    else:
        print(f"  {gene}: Not found in top features")
```

Feature Ranking Code *Figure 6: Code snippet for feature ranking using Gini importance. Shows extraction of feature importances, sorting, and checking for ground truth genes (COL5A2, NDNF, FAT1).*

7.3 Top 10 Ranked Features

Table 5: Top 10 Features by Gini Importance *Features ranked by Gini importance from the best trained Random Forest model (Method 1). Importance values represent the total decrease in node impurity weighted by probability of reaching that node, averaged over all trees.*

Rank	Feature Name	Importance Value
1	COL5A2.1	0.19418
2	COL5A2	0.17763
3	COL5A2.2	0.15805
4	NDNF.2	0.06227
5	SST	0.05203

Rank	Feature Name	Importance Value
6	NDNF.1	0.04587
7	MYO16	0.04039
8	NDNF	0.03645
9	FAT1	0.02484
10	CSGALNACT1.3	0.01194

7.4 Discussion

7.4.1 Comparison with Ground Truth

According to the biological research paper, the i1 cluster selectively expresses: - **COL5A2** (and COL5A2 family genes) - **NDNF** - **FAT1**

Analysis:

The feature ranking results show excellent alignment with the biological ground truth established in the research paper:

1. **COL5A2 Family:** All three COL5A2 variants appear in the top 3 positions:
   - COL5A2.1: Rank 1 (Importance: 0.19418)
   - COL5A2: Rank 2 (Importance: 0.17763)
   - COL5A2.2: Rank 3 (Importance: 0.15805)

   Combined, the COL5A2 family accounts for 0.52986 (52.99%) of the total importance in the top 10 features, demonstrating that these genes are the most critical for i1 cluster classification.

2. **NDNF Family:** All three NDNF variants appear in the top 10:
   - NDNF.2: Rank 4 (Importance: 0.06227)
   - NDNF.1: Rank 6 (Importance: 0.04587)
   - NDNF: Rank 8 (Importance: 0.03645)

   Combined, the NDNF family accounts for 0.14459 (14.46%) of the total importance in the top 10 features.

3. **FAT1:** Appears at Rank 9 (Importance: 0.02484), confirming its importance in i1 cluster identification.

Conclusion: The Random Forest model successfully identified all three ground truth gene families as the most important features, with COL5A2 variants dominating the ranking. This strong alignment with biological knowledge validates that the model is learning meaningful patterns related to i1 cluster biology rather than spurious correlations.

7.4.2 Feature Dominance Analysis

The feature ranking shows a clear dominance pattern:

Highly Dominant Features: - The top 3 features (all COL5A2 variants) account for 0.52986 (52.99%) of the total importance in the top 10 - The top 5 features account for approximately 0.69416 (69.42%) of the total importance - The top 10 features account for the majority of predictive power

Distribution Pattern: - There is a clear drop-off in importance after the top 3 features (COL5A2 family) - The NDNF family (ranks 4, 6, 8) shows moderate importance - FAT1 (rank 9) and other features show lower but still significant importance - The importance values decrease rapidly: from 0.19418 (rank 1) to 0.01194 (rank 10), a 16-fold decrease

Implications: This dominance pattern suggests that: 1. A small number of genes (primarily COL5A2 variants) are highly predictive of i1 cluster classification 2. The model relies heavily on these top features, which aligns with biological knowledge 3. There is a clear hierarchy of feature importance rather than uniform distribution 4. The dominance of COL5A2 variants suggests they are the primary biological markers for i1 cluster identification

7.4.3 Production Feature Selection Recommendation

Based on the feature ranking analysis, I recommend the following feature selection strategy for production:

Option 1: Top 3 Features (COL5A2 Family Only) - **Features:** COL5A2.1, COL5A2, COL5A2.2 - **Cumulative Importance:** ~53% of top 10 importance - **Pros:** Minimal feature set, fastest inference, simplest model - **Cons:** May lose some accuracy, ignores NDNF and FAT1 which are biologically important - **Recommendation:** Not recommended - loses biological context

Option 2: Top 5 Features (COL5A2 + NDNF.2 + SST) - **Features:** Top 5 features including COL5A2 family, NDNF.2, and SST - **Cumulative Importance:** ~69% of top 10 importance - **Pros:** Good balance, includes key biological markers - **Cons:** SST is not a ground truth gene, may not be as biologically relevant - **Recommendation:** Moderate - good for speed but may include non-biological features

Option 3: Top 9 Features (All Ground Truth Genes) - **Features:** All COL5A2 variants, all NDNF variants, FAT1, plus SST and MYO16 - **Cumulative Importance:** ~95% of top 10 importance - **Pros:** Includes all ground truth genes, maintains high accuracy - **Cons:** Slightly more features than minimum - **Recommendation: Recommended** - balances biological relevance with model performance

Option 4: Top 10-20 Features - **Features:** Top 10-20 ranked features - **Pros:** Maximum accuracy retention - **Cons:** More features, slower inference - **Recommendation:** Use if accuracy is critical and inference speed is not a concern

Final Recommendation: For production use, I recommend **Option 3 (Top 9 features)** because: 1. It includes all biologically validated ground truth genes (COL5A2 family, NDNF family, FAT1) 2. It captures ~95% of the importance from the top 10 features 3. It provides a good balance between model simplicity and accuracy retention 4. It maintains biological

interpretability while reducing feature count from 608 to 9 (98.5% reduction) 5. The slight accuracy trade-off (if any) is acceptable given the significant simplification

However, I would recommend validating this choice by retraining the model with only these 9 features and comparing the accuracy with the full model to ensure the performance drop is acceptable for the specific production requirements.

---

8. RF Run Time Test

8.1 Runtime Engine Creation

The runtime engine was created using the best trained Random Forest model from Method 1 (OOB method) with the optimal hyperparameters.

Key Hyperparameters:

Hyperparameter	Value
n_estimators	300
max_features	0.3
max_depth	20
min_samples_split	5
min_samples_leaf	1
class_weight	balanced

8.2 Key Code Snippet

```python

# Use best model from Method 1 for runtime prediction
runtime_rf = best_rf_oob

# Predict on verification samples
y_verification_pred = runtime_rf.predict(X_verification)
y_verification_proba = runtime_rf.predict_proba(X_verification)

print(f"\nVerification Sample Predictions:")
for i in range(len(verification_db)):
    true_label = y_verification.iloc[i]
    pred_label = y_verification_pred[i]
    proba = y_verification_proba[i]

    print(f"\nSample {i+1}:")
    print(f"  True Label: {true_label} ({'i1 cluster' if true_label == 1 else 'non-i1'})")
    print(f"  Predicted Label: {pred_label} ({'i1 cluster' if pred_label == 1 else 'non-i1'})")
    print(f"  Probability Class 0: {proba[0]:.5f}")
    print(f"  Probability Class 1: {proba[1]:.5f}")
    print(f"  Correct: {'Yes' if true_label == pred_label else 'No'}")
    print(f"  Confidence: {max(proba):.5f}")

```

Runtime Prediction Code *Figure 7: Code snippet for runtime prediction on verification database. Shows prediction and probability computation for each verification sample.*

8.3 Verification Database Predictions

Table 6: Runtime Prediction Results on Verification Database *Predictions made by the best trained Random Forest model (Method 1) on the 2 verification samples. Probabilities shown to 5 decimal places.*

Sample	True Label	Predicted Label	Prob(Class 0)	Prob(Class 1)	Correct?
1	0 (non-i1)	0 (non-i1)	0.83333	0.16667	Yes
2	1 (i1 cluster)	1 (i1 cluster)	0.03667	0.96333	Yes

8.4 Discussion

8.4.1 Classification Accuracy

Both verification samples were correctly classified by the Random Forest model:

- **Sample 1:** Correctly predicted as class 0 (non-i1 cluster)
- **Sample 2:** Correctly predicted as class 1 (i1 cluster)

This represents a 100% accuracy rate on the verification set (2/2 correct predictions). While this is a very small sample size, it provides initial validation that the model can generalize to unseen data. The correct classification of both samples, including one from each class, suggests the model has learned meaningful patterns that distinguish i1 cluster samples from non-i1 samples.

8.4.2 Confidence Assessment

For each sample, discuss: - **Confidence level:** Based on the predicted class probability - **How confidence was determined:** The maximum probability between the two classes - **Interpretation:** - High confidence (>0.8): Very confident prediction - Medium confidence (0.6-0.8): Moderate confidence - Low confidence (<0.6): Less confident, may need review

Sample 1 Analysis: - **True Label:** 0 (non-i1 cluster) - **Predicted Label:** 0 (non-i1 cluster) ✓ Correct - **Class Probabilities:** P(Class 0) = 0.83333, P(Class 1) = 0.16667 - **Confidence Level:** 0.83333 (83.33%) - **High Confidence** - **Interpretation:** The model is highly confident (83.33%) that this sample belongs to the non-i1 class. The probability of 0.16667 for class 1 indicates a low likelihood of being an i1 cluster sample. This is a high-confidence prediction, well above the 0.8 threshold for very confident predictions.

Sample 2 Analysis: - **True Label:** 1 (i1 cluster) - **Predicted Label:** 1 (i1 cluster) ✓ Correct - **Class Probabilities:** P(Class 0) = 0.03667, P(Class 1) = 0.96333 - **Confidence Level:** 0.96333 (96.33%) - **Very High Confidence** - **Interpretation:** The model is extremely confident (96.33%) that this sample belongs to the i1 cluster class. The very low probability of 0.03667 for class 0 indicates an almost certain classification as i1 cluster. This is a very high-confidence prediction, significantly above the 0.8 threshold, suggesting the model has strong evidence (likely high expression of COL5A2, NDNF, and/or FAT1 genes) for this classification.

Overall Confidence Assessment: Both predictions show high to very high confidence levels (83.33% and 96.33%), well above the 0.6 threshold for acceptable confidence. The confidence is determined by taking the maximum probability between the two classes, which represents the proportion of trees in the Random Forest ensemble that voted for that class.

For Sample 1: The probability of 0.83333 for class 0 means that approximately 83.33% of the 300 trees in the ensemble voted for class 0 (non-i1), indicating strong agreement among trees.

For Sample 2: The probability of 0.96333 for class 1 means that approximately 96.33% of the 300 trees voted for class 1 (i1 cluster), indicating very strong agreement. This high confidence suggests the model has strong evidence (likely high expression of COL5A2, NDNF, and/or FAT1 genes) for this classification.

The fact that both samples were correctly classified with high confidence suggests the model is not only accurate but also reliable in its predictions, with the majority of trees agreeing on the classification.

---

9. Resources

9.1 References

1. Aevermann B., Novotny M., Bakken T., Miller J., Diehl A., Osumi-Sutherland D., Lasken R., Lein E., Scheuermann R.: "Cell type discovery using single cell transcriptomics: implications for ontological representation", Human Molecular Genetics 27(R1): R40-R47 · March 2018

2. scikit-learn Documentation: https://scikit-learn.org/stable/

3. [Add any other references used]

9.2 GenAI Usage

GenAI was used as a coding assistant and tutor for this assignment. See Appendix I for detailed usage information.

9.3 People Who Helped

None - this was individual work as required by the assignment.

Appendix I: GenAI Use Summary

GenAI (Cursor AI Assistant) was used as a coding assistant and tutor throughout this assignment. Below are the specific tasks where GenAI was utilized:

Task 1: Creating the Complete RF Pipeline Script

- **GenAI Tool Used:** Cursor AI Assistant (Auto)
- **How it helped:** Generated the complete Python script (`hw2_rf_pipeline.py`) implementing all required components: database audit, data splitting, OOB estimation, manual 3-fold CV, feature ranking, and runtime prediction. The GenAI assistant provided well-structured, documented code with proper error handling and output formatting.
- **Example Prompts:**

  ```
  "help me make sure i get full points in this assignment: CSC 659 859 Sp
  ring 2026 HW 2 –ML pipeline experiment with Random Forest"
  "can you fill in the report template for me?"
  ```

- **Key GenAI Output:**
  - Complete Python script with all ML pipeline components
  - Report template with all sections
  - Checklist for ensuring full points
- **Usefulness Rating:** 5 (Very useful) - The GenAI assistant was essential for creating a comprehensive, well-structured solution that addresses all assignment requirements.

Task 2: Understanding Assignment Requirements

- **GenAI Tool Used:** Cursor AI Assistant
- **How it helped:** Helped clarify assignment requirements, especially regarding manual CV implementation (not using one-liners), accuracy measure formatting (5 decimal places), and ground truth gene comparison requirements.
- **Example Prompts:**

  ```
  "What does manual 3-fold CV mean? Can I use sklearn's cross_val_score?"
  "What accuracy measures do I need to report?"
  ```

- **Key GenAI Output:**
  - Clarification that manual CV means implementing fold splitting and training manually
  - List of required accuracy measures (accuracy, precision, recall, F1)
  - Explanation of ground truth genes (COL5A2, NDNF, FAT1)
- **Usefulness Rating:** 4 (Very useful) - Helped ensure all requirements were understood and met.

Task 3: Code Debugging and Optimization

- **GenAI Tool Used:** Cursor AI Assistant

- **How it helped:** Assisted with debugging import errors (sklearn module not found) and optimizing the grid search parameters for reasonable execution time.
- **Example Prompts:**

  ```
  "ModuleNotFoundError: No module named 'sklearn'"
  "The grid search is taking too long, how can I reduce it?"
  ```

- **Key GenAI Output:**
  - Solution to install scikit-learn package
  - Suggestion to use reduced parameter grid
- **Usefulness Rating:** 3 (Moderately useful) - Standard debugging assistance.

Task 4: Report Writing and Analysis

- **GenAI Tool Used:** Cursor AI Assistant
- **How it helped:** Filled in the report template with actual results, generated analysis and discussion sections comparing methods, interpreting feature rankings, and assessing prediction confidence.
- **Example Prompts:**

  ```
  "can you fill in the report template for me?"
  ```

- **Key GenAI Output:**
  - Complete report with all sections filled in
  - Detailed analysis comparing Method 1 vs Method 2
  - Feature ranking interpretation and ground truth comparison
  - Confidence assessment for runtime predictions
- **Usefulness Rating:** 5 (Very useful) - Significantly reduced time needed for report writing while ensuring comprehensive coverage of all requirements.

Task 5: Final Review and Critical Requirements Check

- **GenAI Tool Used:** Google Gemini
- **How it helped:** Reviewed the draft report template against Professor Petkovic's strict requirements and identified critical areas that needed attention, including sample-to-feature ratio discussion, manual CV verification, 5-decimal formatting, and submission requirements.
- **Example Prompts:**

  ```
  "I have reviewed your draft template against Professor Petkovic's stric
  t requirements..."
  ```

- **Key GenAI Output:**

  - Checklist of critical technical details to verify
  - Specific requirements for manual CV implementation
  - File naming and submission format requirements
  - Reminders about 5-decimal precision and confusion matrix formatting
- **Usefulness Rating:** 5 (Very useful) - Provided critical quality assurance review that helped ensure all assignment requirements would be met for full points.

Overall Assessment: GenAI was used primarily as a coding assistant and tutor, helping to structure the solution, understand requirements, and generate well-documented code. All code and analysis were reviewed and verified for accuracy. The GenAI assistants (Cursor AI and Google Gemini) were particularly valuable for ensuring all assignment requirements were met and for generating comprehensive documentation and analysis.

Appendix II: Full Code

The complete code is provided in the file `hw2_rf_pipeline.py`. For submission, this code should be included in PDF format. The code includes:

- **Header comments:** Purpose, HW 2 information, author name (Nathaniel Moreno), date
- **In-line documentation:** Comments explaining each major section
- **Clear organization:** Code organized into logical sections matching the report structure

Key sections of the code: 1. Database loading and audit (Section 1) 2. Training/verification database creation (Section 2) 3. Method 1: OOB accuracy estimation with grid search (Section 3) 4. Method 2: Manual 3-fold cross-validation (Section 4) 5. Feature ranking analysis (Section 5) 6. Runtime prediction on verification database (Section 6) 7. Results saving and summary (Section 7)

The code is well-documented with comments explaining the purpose of each section, key parameters, and methodology. All random seeds are set to 42 for reproducibility.

Note: For the PDF submission, the code from `hw2_rf_pipeline.py` should be copied and formatted appropriately in the PDF document. The code header includes: "Author: Nathaniel (Nate) Moreno" as required.

Submission Checklist

Before submitting, verify the following:

- ☐ **File Name:** `CSC 659-859 Spring 2026 HW 2 Moreno.PDF` (exact format with spaces and capitalization)
- ☐ **Email To:** Petkovic@sfsu.edu

- ☐ **Email Subject:** `CS 659 859 Spring 2026 HW 2 Moreno` (Note: "CS" not "CSC", no dashes in subject)
- ☐ **Email Body:** Includes courtesy text
- ☐ **Title Page:** Name is "Nathaniel (Nate) Moreno" (not just "Nathaniel Moreno")
- ☐ **All Sections:** Complete with all required information
- ☐ **Code Header:** Appendix II code has header with "Author: Nathaniel (Nate) Moreno"
- ☐ **5 Decimal Places:** All accuracy measures formatted to exactly 5 decimal places
- ☐ **Confusion Matrices:** Show full counts (not percentages), axes labeled
- ☐ **Manual CV:** Clearly stated that CV was done manually (not using sklearn's built-in methods)
- ☐ **Sample-to-Feature Ratio:** Explicitly discussed as a limitation (1.43:1 does not meet 10:1 requirement)
- ☐ **Ground Truth Comparison:** COL5A2, NDNF, FAT1 discussed in feature ranking section
- ☐ **Runtime Probabilities:** Both verification samples show class probabilities
- ☐ **Appendix I:** GenAI usage documented with prompts and usefulness ratings
- ☐ **Appendix II:** Full code included in PDF format